

Chapter 13

Abstract Class & Interface

Motivation

- In programming, different objects may share similar behaviors but are not necessarily related by inheritance.
- How can we ensure consistency across multiple classes while allowing flexibility in implementation?
- What if a class needs to inherit behavior from multiple sources?

Objectives

- To design and use abstract classes (§13.2).
- To generalize numeric wrapper classes, **BigInteger**, and **BigDecimal** using the abstract **Number** class (§13.3).
- To process a calendar using the **Calendar** and **GregorianCalendar** classes (§13.4).
- To specify common behavior for objects using interfaces (§13.5).
- To define interfaces and define classes that implement interfaces (§13.5).
- To define a natural order using the **Comparable** interface (§13.6).
- To make objects cloneable using the **Cloneable** interface (§13.7).
- To explore the similarities and differences among concrete classes, abstract classes, and interfaces (§13.8).
- To design the **Rational** class for processing rational numbers (§13.9).
- To design classes that follow the class-design guidelines (§13.10).

Object-Oriented Programming (OOP)

E

A

I

P

Object-Oriented Programming (OOP)

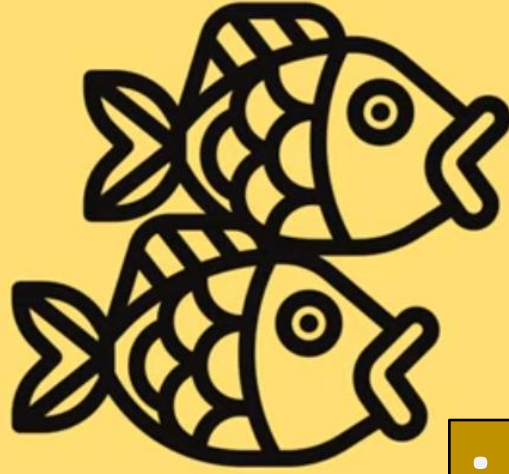
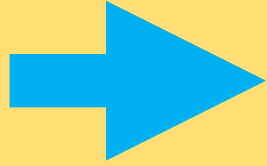
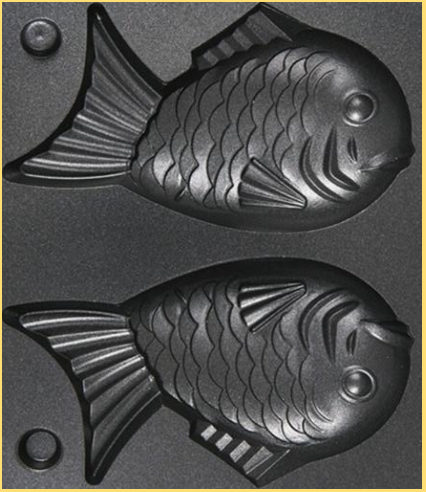
Encapsulation

Abstraction

Inheritance

Polymorphism

Concrete Class



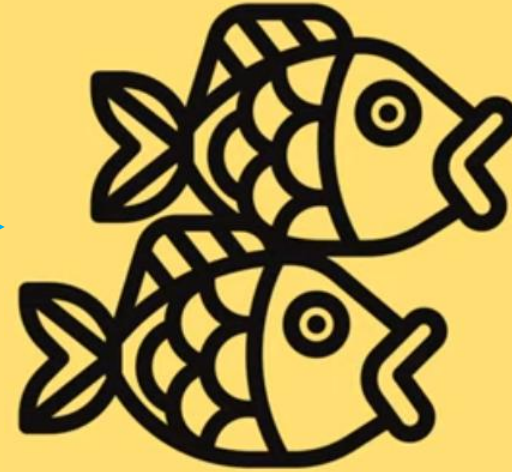
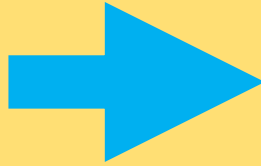
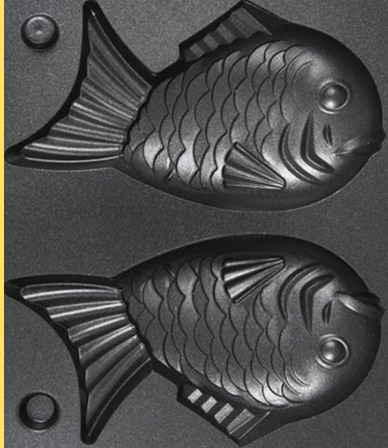
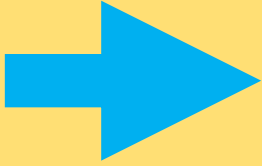
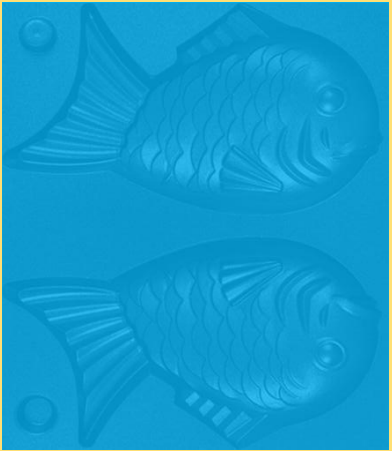
instantiation

```
Scanner kInput = new Scanner(System.in);  
FISH myFish = new FISH();
```

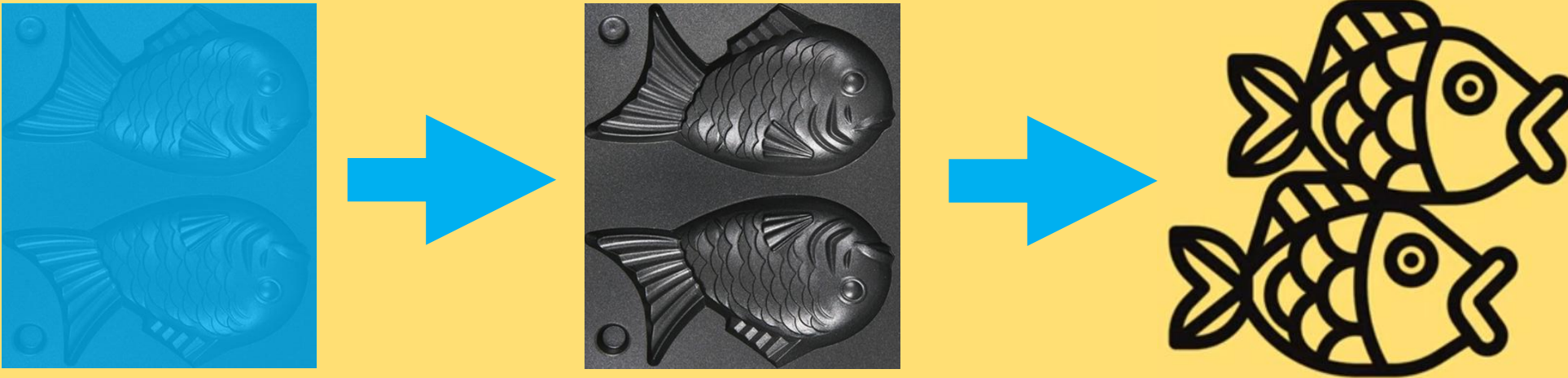
Fish	
name	
age	
swim()	

- **template** for object creation
- **properties** – states, fields, attributes, variables
- **methods** – behaviors of an object
- **user-defined** data type

Abstract Class



Abstract Class



- **template** for class
- **abstract method** - signature only **without a body**
- **subclass** must implement all the **abstract methods**

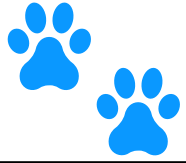
No instantiation

Abstract class



concrete class

Abstract Class



Mammal

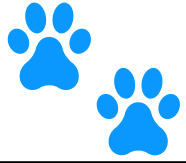
```
//abstract class declaration
: class Mammal {

    //properties

    //concrete methods
    public void say(String name){
        System.out.println("I am " + name);
    }

    //abstract methods
    public abstract void run();
    public abstract void eat();
}
```

Abstract Class



Mammal

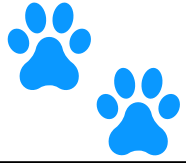
```
//abstract class declaration
abstract class Mammal {

    //properties
    private String name;

    //concrete methods
    public void say(String name){
        System.out.println("I am " + name);
    }

    //abstract methods
    public abstract void run();
    public abstract void eat();
}
```

Abstract Class



Mammal

//abstract class declaration

```
abstract class Mammal {
```

//properties

```
private String name;
```

//concrete methods

```
public void say(String name){  
    System.out.println("I am " + name);  
}
```

//abstract methods

```
public abstract void run();  
public abstract void eat();
```

```
}
```



Dog



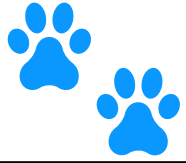
//concrete class declaration

```
class Dog extends Mammal {
```

//the methods should be implemented

```
}
```

Abstract Class



Mammal

//abstract class declaration

```
abstract class Mammal {
```

//properties

```
private String name;
```

//concrete methods

```
public void say(String name){  
    System.out.println("I am " + name);  
}
```

//abstract methods

```
public abstract void run();  
public abstract void eat();
```

```
}
```



Dog



//concrete class declaration

```
class Dog extends Mammal {
```

//the methods should be implemented
@Override

```
public void run(){  
    System.out.println("running!");  
}  
public void eat(){  
    System.out.println("eating!");  
}  
}
```

Abstract Class

[illegible]

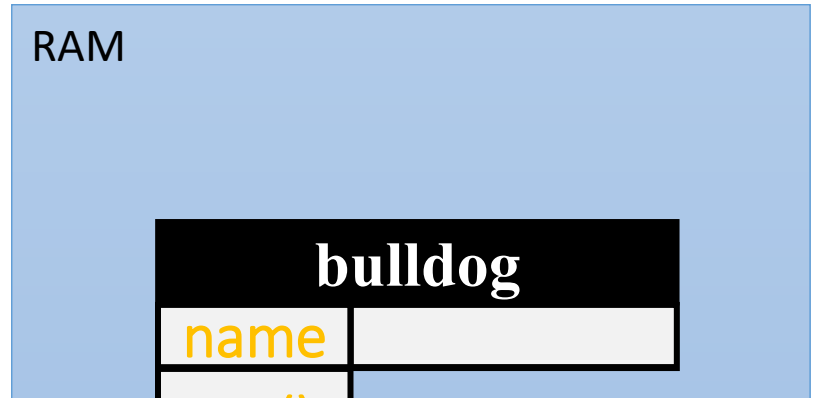
Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
  
  
  
  
  
  
    }  
}
```



Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
    }  
}
```



@Override

```
public void run(){  
    System.out.println("running!");  
}  
  
public void eat(){  
    System.out.println("eating!");  
}
```

Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
    }  
}
```

```
//concrete methods  
public void say(String name){  
    System.out.println("I am " + name);  
}
```

@Override

```
public void run(){  
    System.out.println("running!");  
}  
  
public void eat(){  
    System.out.println("eating!");  
}
```


Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
  
    }  
}
```

```
//concrete methods  
public void say(String name){  
    System.out.println("I am " + name);  
}
```

@Override

```
public void run(){  
    System.out.println("running!");  
}  
public void eat(){  
    System.out.println("eating!");  
}
```

I am Billy

Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
        bulldog.run();  
    }  
}
```

```
//concrete methods  
public void say(String name){  
    System.out.println("I am " + name);  
}
```

@Override

```
public void run(){  
    System.out.println("running!");  
}  
public void eat(){  
    System.out.println("eating!");  
}
```

I am Billy

Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
        bulldog.run();  
        bulldog.eat();  
    }  
}
```

```
//concrete methods  
public void say(String name){  
    System.out.println("I am " + name);  
}
```

@Override

```
public void run(){  
    System.out.println("running!");  
}  
public void eat(){  
    System.out.println("eating!");  
}
```

I am Billy
running!

Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
        bulldog.run();  
        bulldog.eat();  
    }  
}
```

```
//concrete methods  
public void say(String name){  
    System.out.println("I am " + name);  
}
```

@Override

```
public void run(){  
    System.out.println("running!");  
}  
public void eat(){  
    System.out.println("eating!");  
}
```

I am Billy
running!
eating!

Abstract Class

```
public class AbstractTest {  
    public static void main(String[] args) {  
        Dog bulldog = new Dog();  
  
        bulldog.say("Billy");  
        bulldog.run();  
        bulldog.eat();  
    }  
}
```



RAM

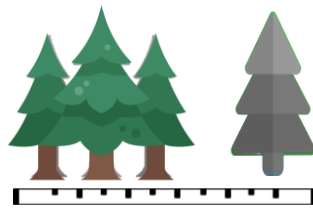
bulldog	
name	
say()	
run()	
eat()	

I am Billy
running!
eating!

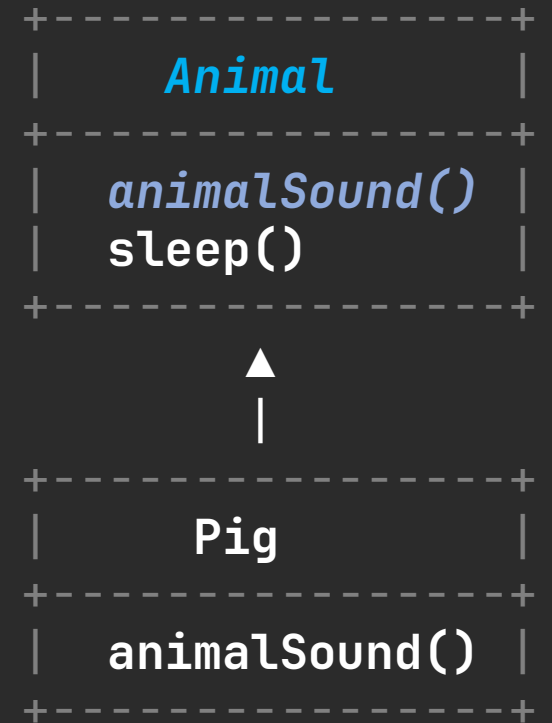
Abstract Class

```
1 class test {  
2     public static void main(String[] args) {  
3         Pig myPig = new Pig();  
4         myPig.animalSound();  
5         myPig.sleep();  
6     }  
7 }  
8  
9 abstract class Animal {  
10     public abstract void animalSound();  
11     public void sleep() {  
12         System.out.println("Zzz");  
13     }  
14 }  
15 class Pig extends Animal {  
16     public void animalSound() {  
17         System.out.println("Wee wee");  
18     }  
19 }
```

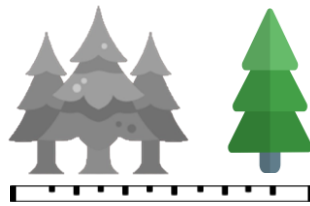
Abstract Class



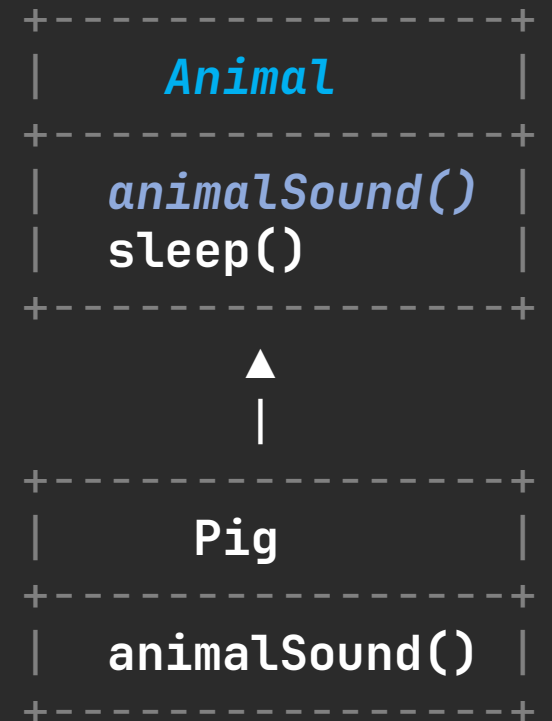
```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 abstract class Animal {
10     public abstract void animalSound();
11     public void sleep() {
12         System.out.println("Zzz");
13     }
14 }
15
16 class Pig extends Animal {
17     public void animalSound() {
18         System.out.println("Wee wee");
19     }
20 }
```



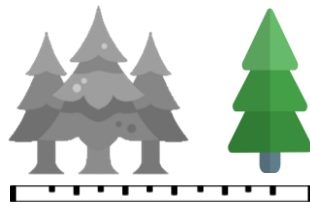
Abstract Class



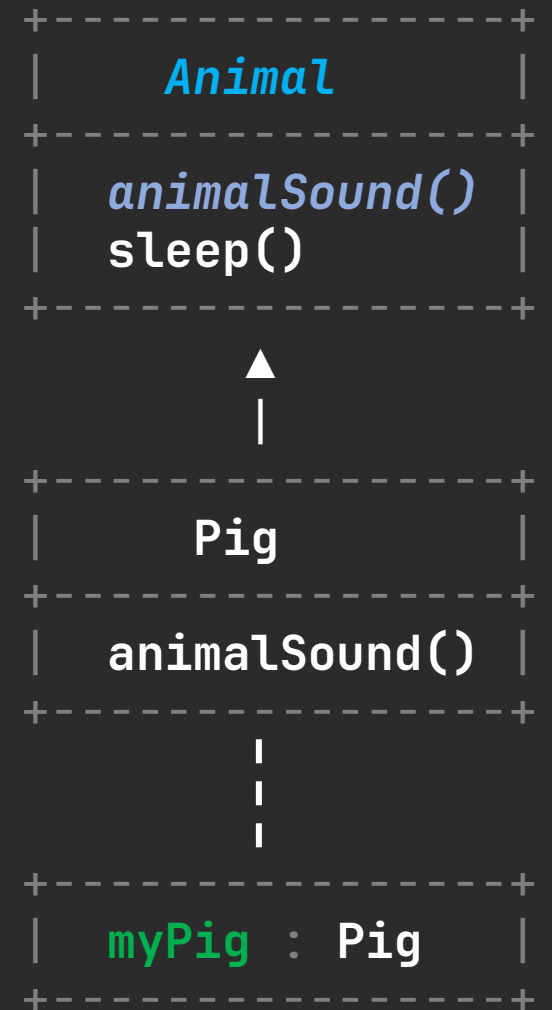
```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig(); 1
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 abstract class Animal {
10     public abstract void animalSound();
11     public void sleep() {
12         System.out.println("Zzz");
13     }
14 }
15 class Pig extends Animal {
16     public void animalSound() {
17         System.out.println("Wee wee");
18     }
19 }
```



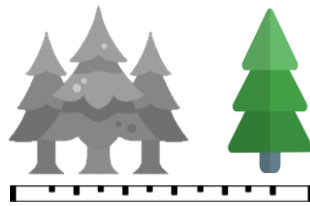
Abstract Class



```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound(); ?
5         myPig.sleep();
6     }
7 }
8
9 abstract class Animal {
10     public abstract void animalSound();
11     public void sleep() {
12         System.out.println("Zzz");
13     }
14 }
15
16 class Pig extends Animal {
17     public void animalSound() {
18         System.out.println("Wee wee");
19     }
20 }
```

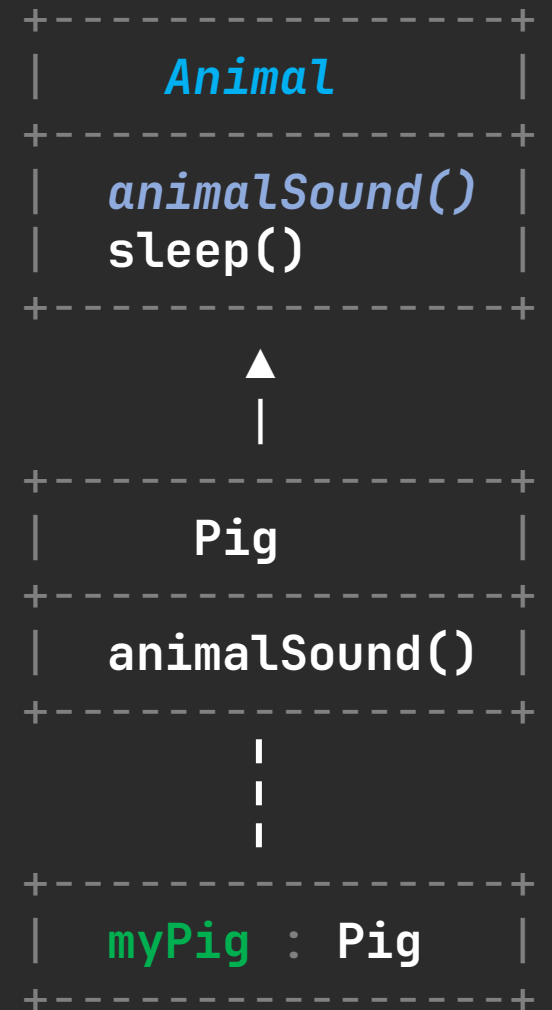


Abstract Class

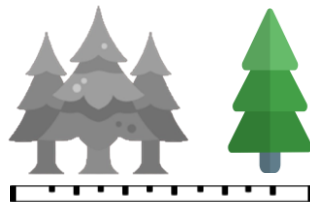


```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 abstract class Animal {
10     public abstract void animalSound();
11     public void sleep() {
12         System.out.println("Zzz");
13     }
14 }
15
16 class Pig extends Animal {
17     public void animalSound() {
18         System.out.println("Wee wee");
19     }
20 }
```

Wee wee

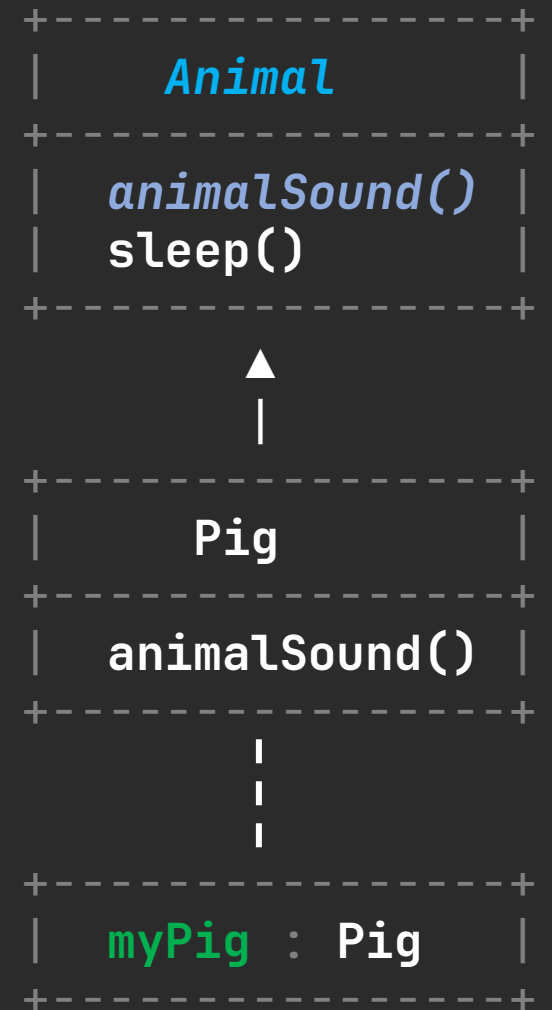


Abstract Class

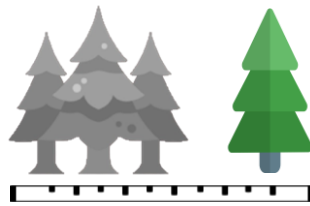


```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 abstract class Animal {
10     public abstract void animalSound();
11     public void sleep() {
12         System.out.println("Zzz");
13     }
14 }
15 class Pig extends Animal {
16     public void animalSound() {
17         System.out.println("Wee wee");
18     }
19 }
```

Wee wee



Abstract Class

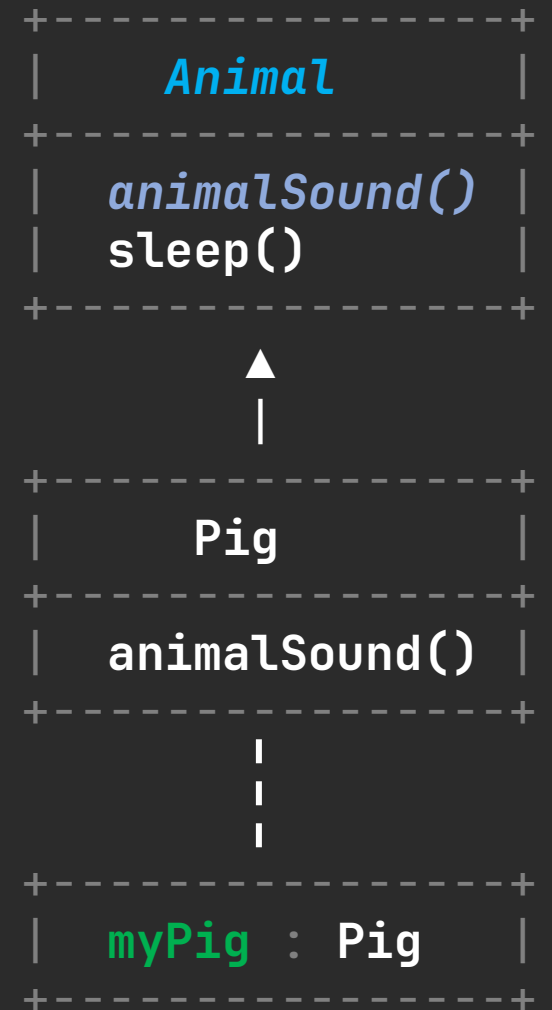


```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
```

Wee wee
Zzz



```
8 abstract class Animal {
9     public abstract void animalSound();
10    public void sleep() {
11        System.out.println("Zzz");
12    }
13 }
14
15 class Pig extends Animal {
16     public void animalSound() {
17         System.out.println("Wee wee");
18     }
19 }
```

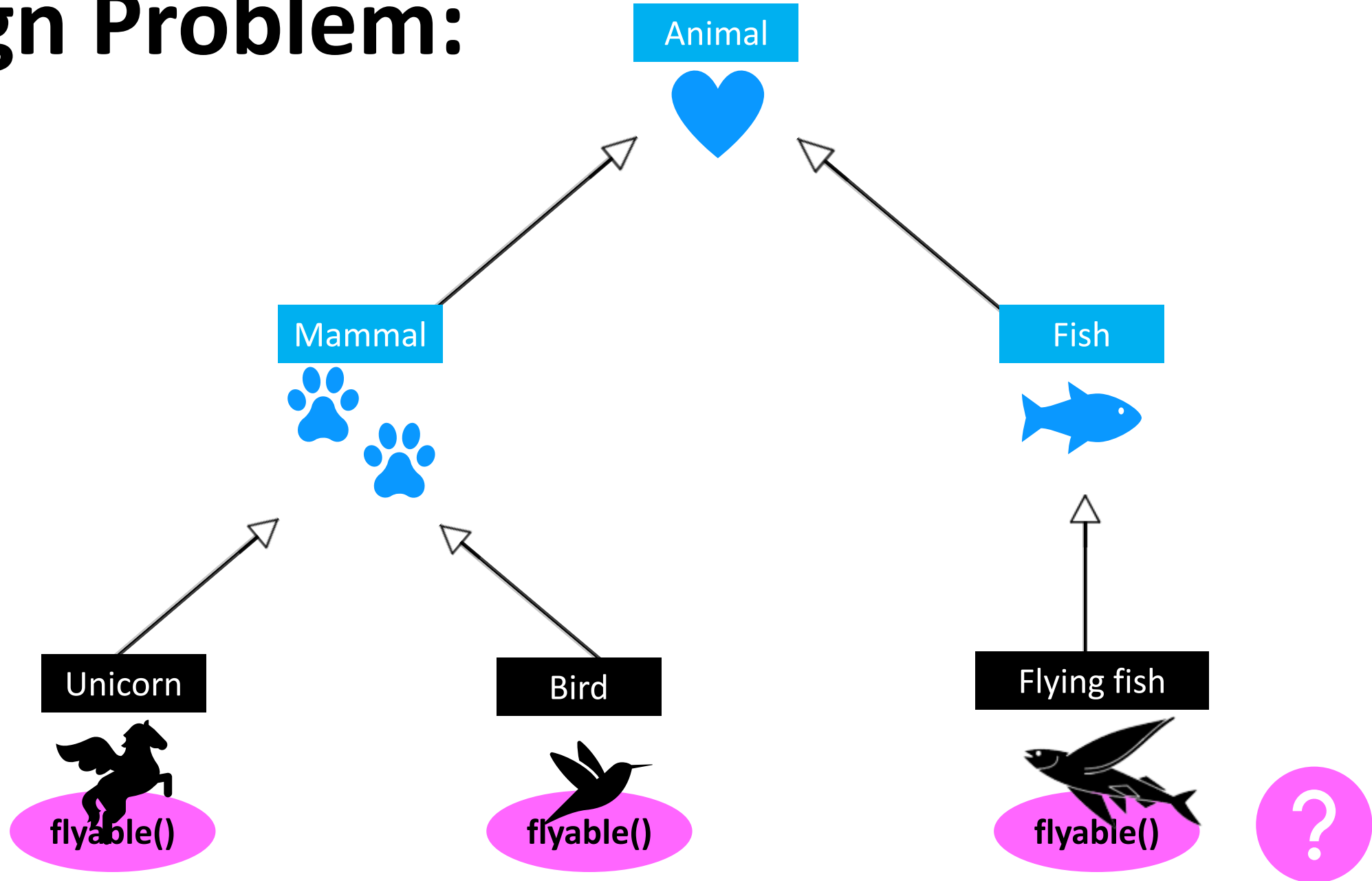


Abstract Class: Work Order

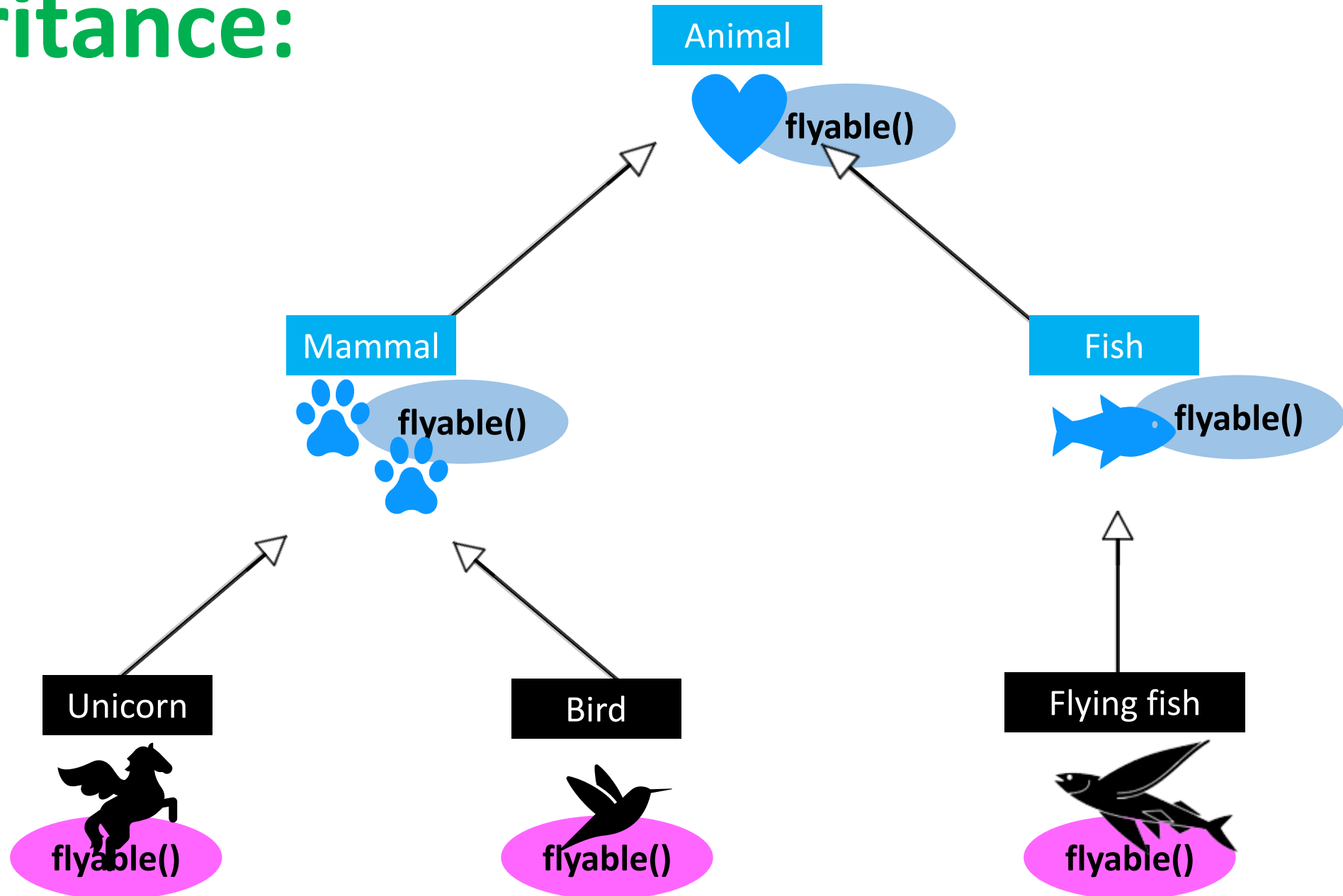


- Provides a **blueprint** for project structure
- Defines **required tasks** (e.g., **abstract methods**)
- Organizes **work** efficiently
- **Delegates** tasks in a structured, professional way.

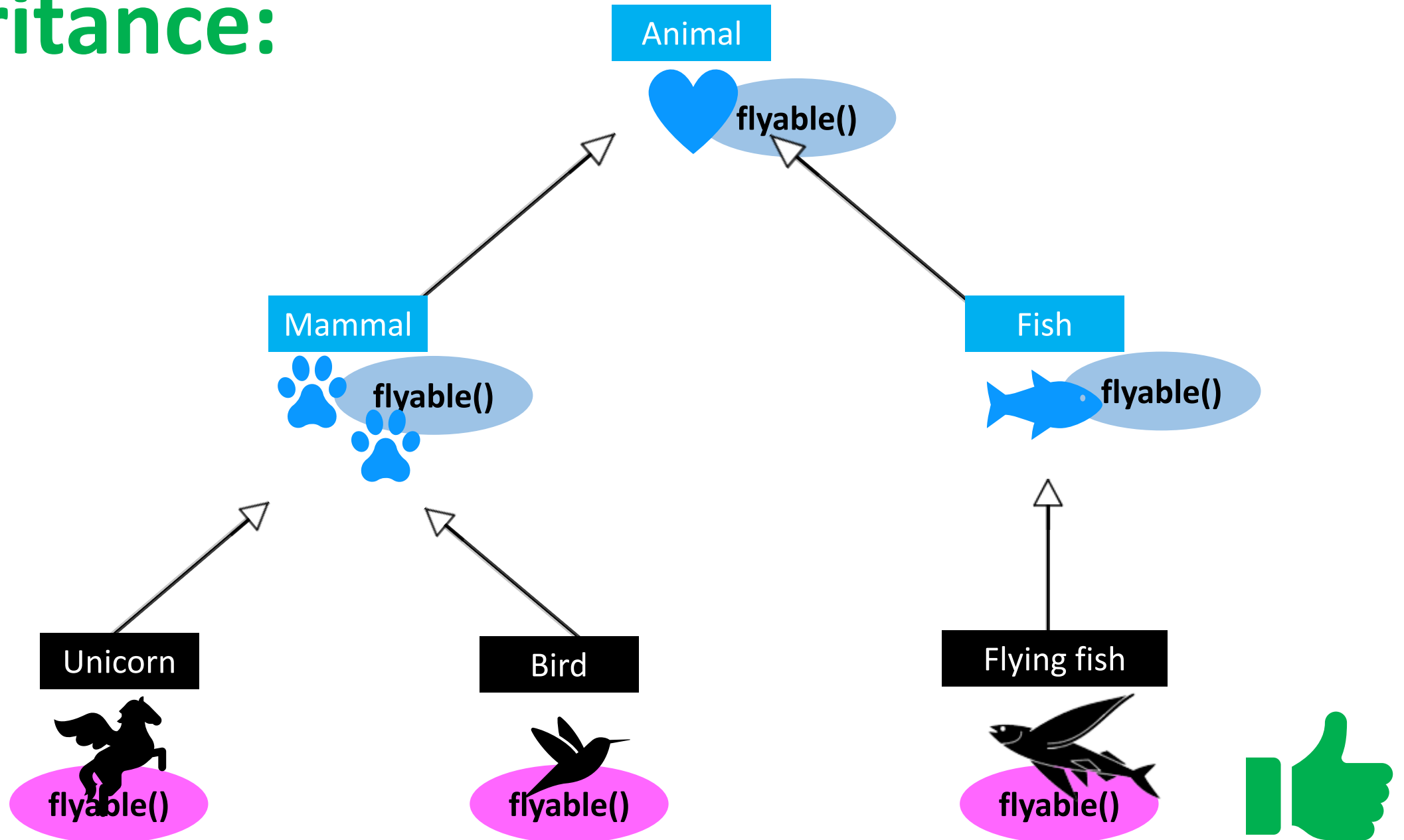
Design Problem:



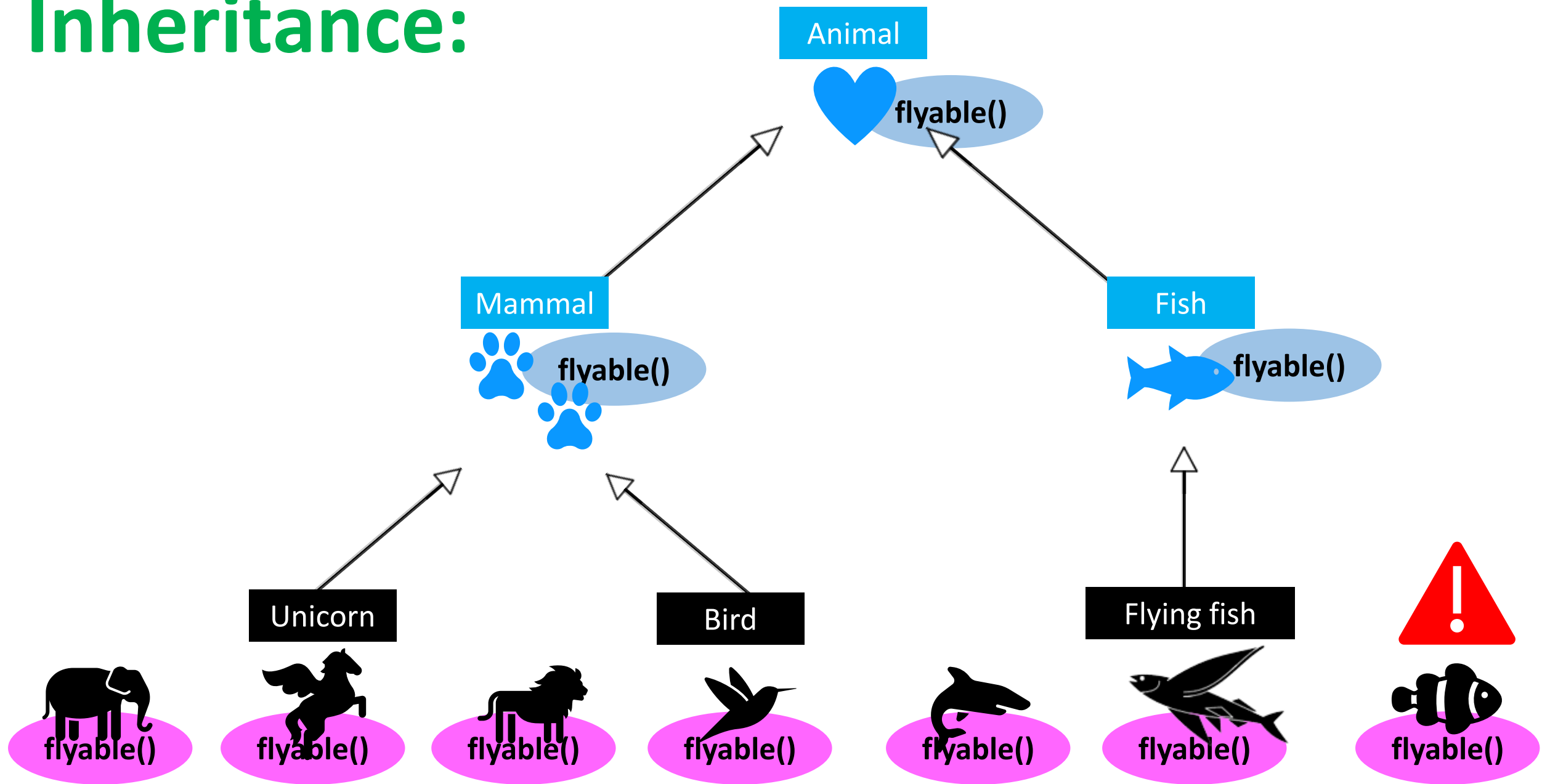
Inheritance:



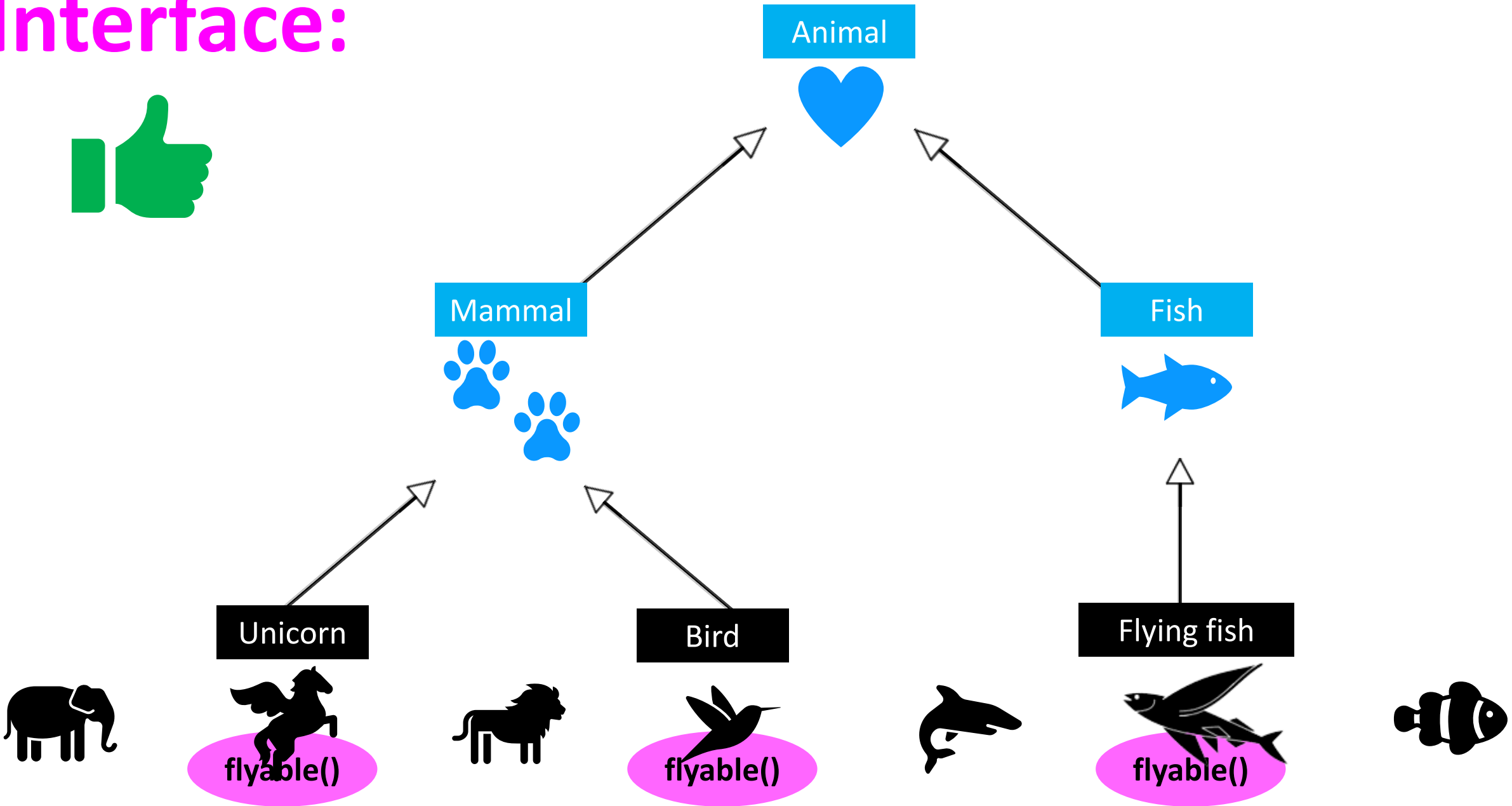
Inheritance:



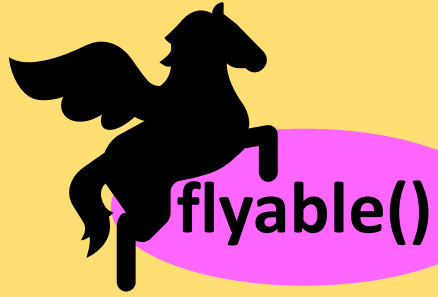
Inheritance:



Interface:



Interface

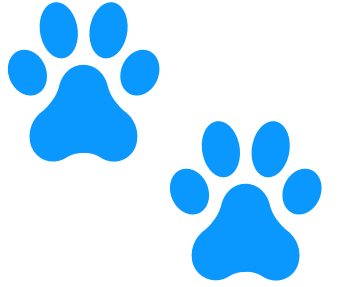


(abstract) class : **extends**
interface : **implements**

- $\approx$ abstract class: **abstract method** + concrete method
- CAN-DO relationship
- subclass must implement all the abstract methods

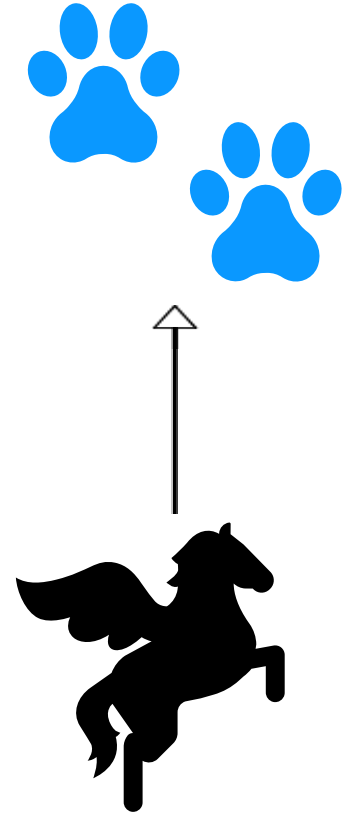
Interface

```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14 public class test {  
15     public static void main(String[] args) {  
16  
17     }  
18 }  
19
```



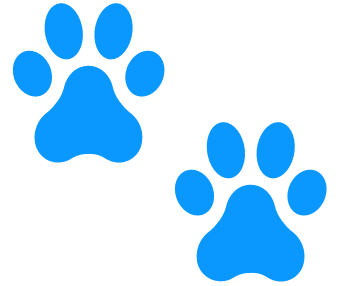
Interface

```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4 class Unicorn extends Mammal {  
5     // Inherits all non-private members from Mammal  
6  
7  
8 }  
9  
10  
11  
12  
13  
14 public class test {  
15     public static void main(String[] args) {  
16  
17     }  
18 }  
19
```



Interface

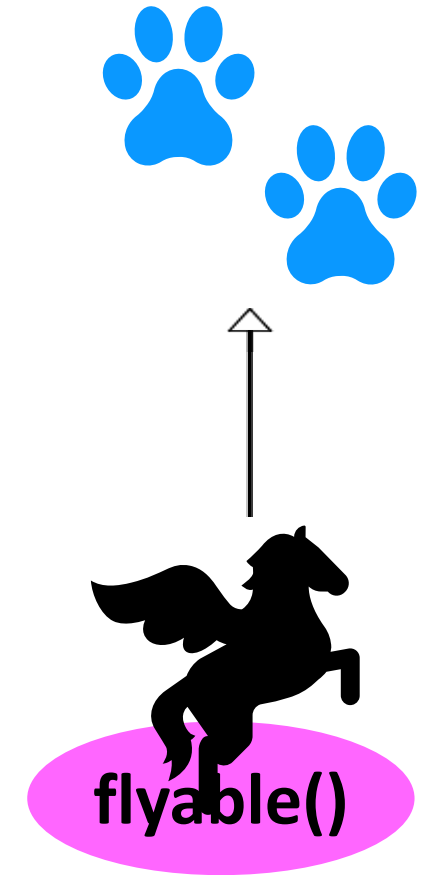
```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4 class Unicorn extends Mammal {  
5     // Inherits all non-private members from Mammal  
6  
7  
8 }  
9  
10 interface Flyable {  
11     void fly();  
12 }  
13  
14 public class test {  
15     public static void main(String[] args) {  
16  
17     }  
18 }  
19
```



flyable()

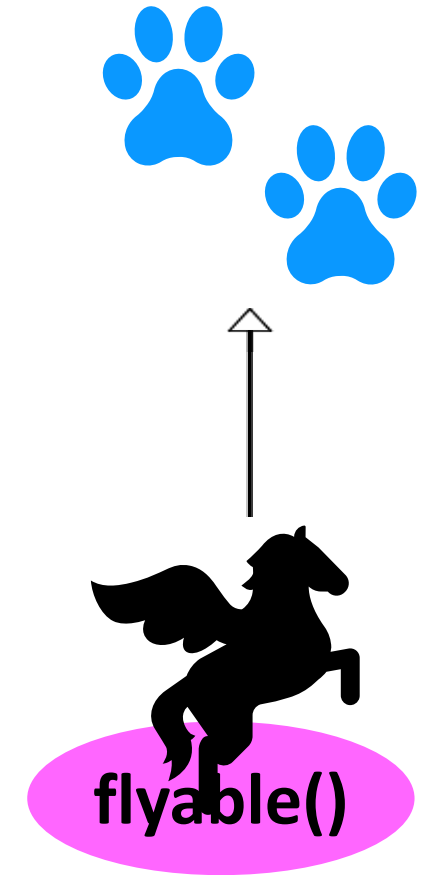
Interface

```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4 class Unicorn extends Mammal implements Flyable {  
5     // Inherits all non-private members from Mammal  
6  
7  
8 }  
9  
10 interface Flyable {  
11     void fly();  
12 }  
13  
14 public class test {  
15     public static void main(String[] args) {  
16  
17     }  
18 }  
19
```



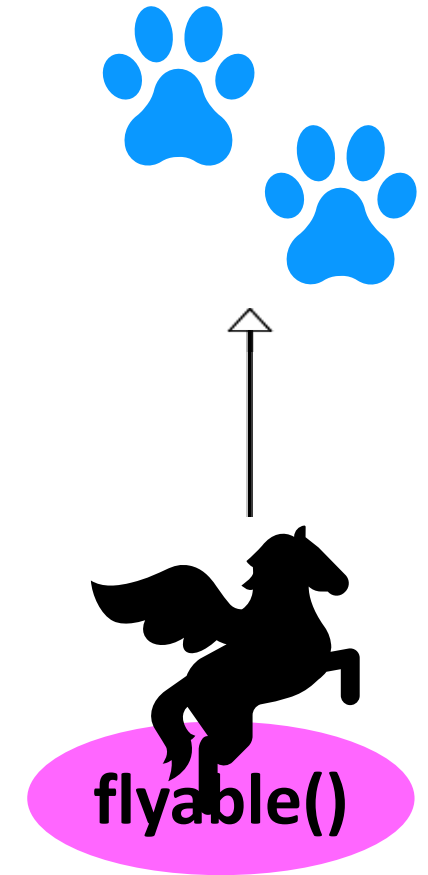
Interface

```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4 class Unicorn extends Mammal implements Flyable {  
5     // Inherits all non-private members from Mammal  
6     @Override  
7     public void fly() {  
8         System.out.println("Unicorn is flying!"); }  
9 }  
10  
11 interface Flyable {  
12     void fly();  
13 }  
14  
15 public class test {  
16     public static void main(String[] args) {  
17  
18     }  
19 }  
20
```



Interface

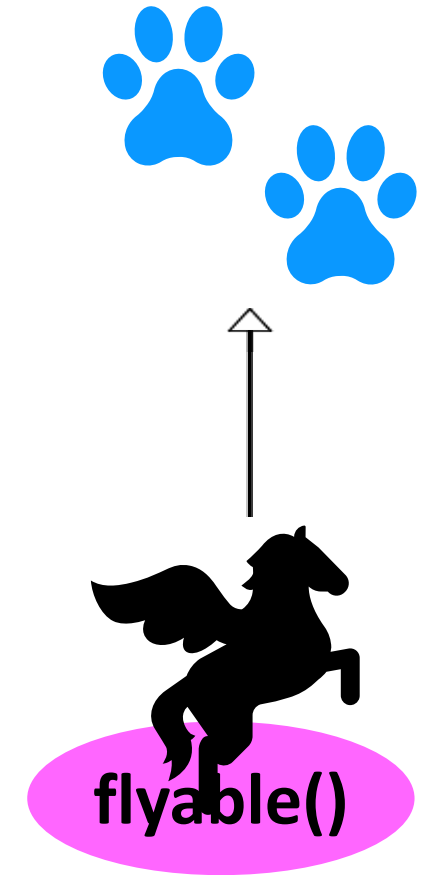
```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4 class Unicorn extends Mammal implements Flyable {  
5     // Inherits all non-private members from Mammal  
6     @Override  
7     public void fly() {  
8         System.out.println("Unicorn is flying!"); }  
9 }  
10  
11 interface Flyable {  
12     void fly();  
13 }  
14  
15 public class test {  
16     public static void main(String[] args) {  
17         Unicorn myUnicorn = new Unicorn();  
18         myUnicorn.fly();  
19     }  
20 }
```



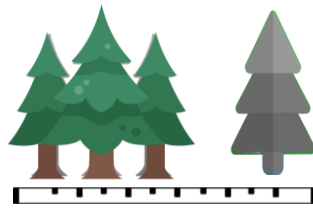
Interface

```
1 abstract class Mammal {  
2     // mammal properties & methods here }  
3  
4 class Unicorn extends Mammal implements Flyable {  
5     // Inherits all non-private members from Mammal  
6     @Override  
7     public void fly() {  
8         System.out.println("Unicorn is flying!"); }  
9 }  
10  
11 interface Flyable {  
12     void fly();  
13 }  
14  
15 public class test {  
16     public static void main(String[] args) {  
17         Unicorn myUnicorn = new Unicorn();  
18         myUnicorn.fly();  
19     }  
20 }
```

Unicorn is flying!

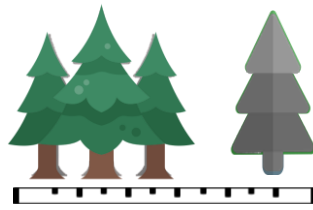


Interface

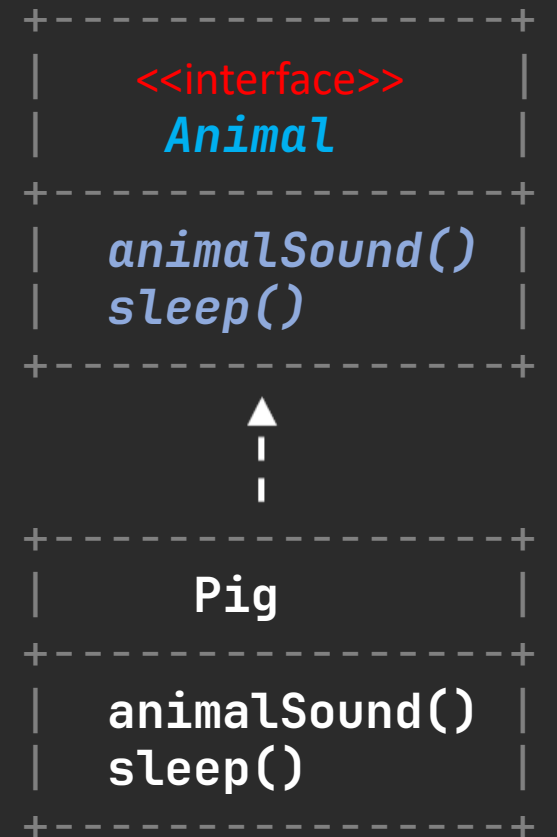


```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");
17     }
18     public void sleep() {
19         System.out.println("Zzz");
20     }
21 }
```

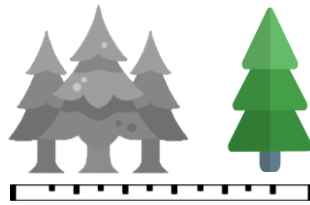
Interface



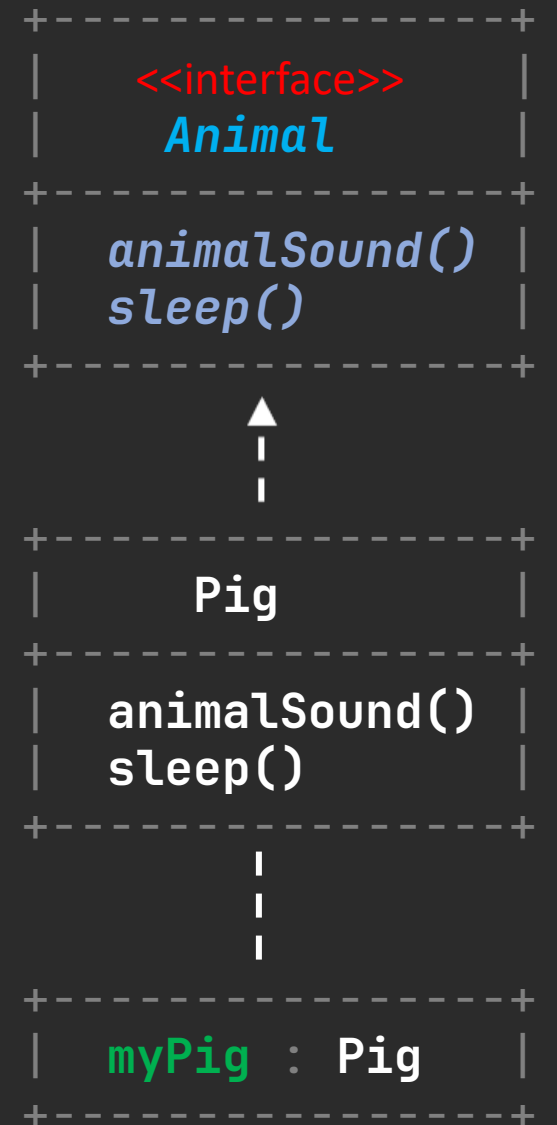
```
1 class test {
2   → public static void main(String[] args) {
3       Pig myPig = new Pig();
4       myPig.animalSound();
5       myPig.sleep();
6   }
7 }
8
9 interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");    }
17     public void sleep() {
18         System.out.println("Zzz");    }
19 }
```



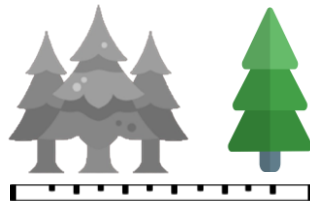
Interface



```
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig(); 1
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");
17     }
18     public void sleep() {
19         System.out.println("Zzz");
20     }
21 }
```



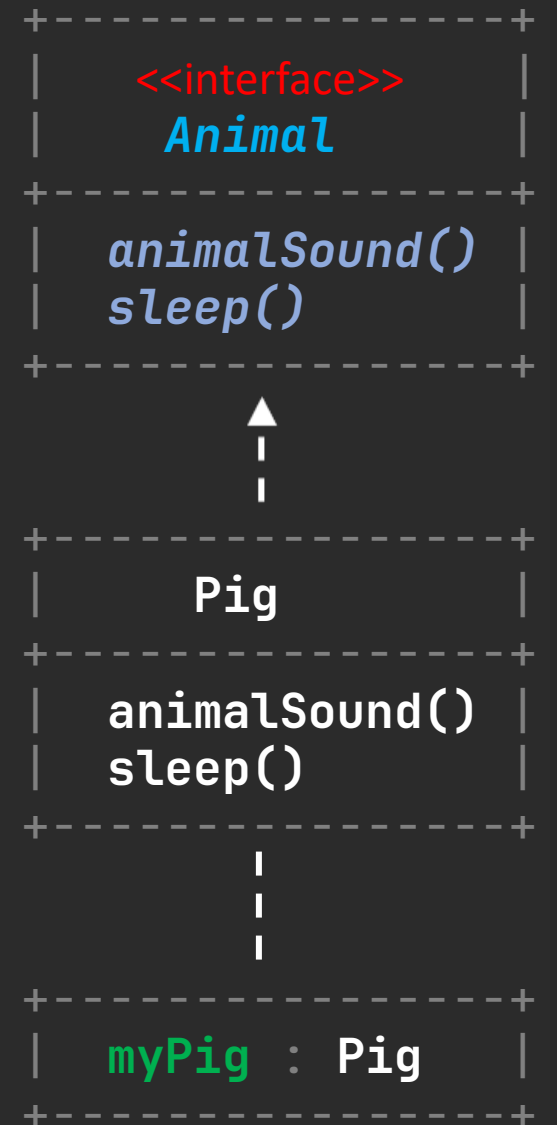
Interface



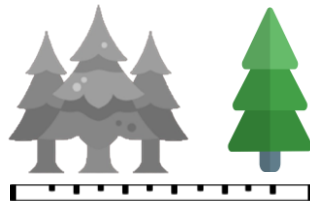
```

1  class test {
2  → public static void main(String[] args) {
3      Pig myPig = new Pig();
4      myPig.animalSound(); ?
5      myPig.sleep();
6  }
7  }
8
9  interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");
17     }
18     public void sleep() {
19         System.out.println("Zzz");
20     }
21 }

```

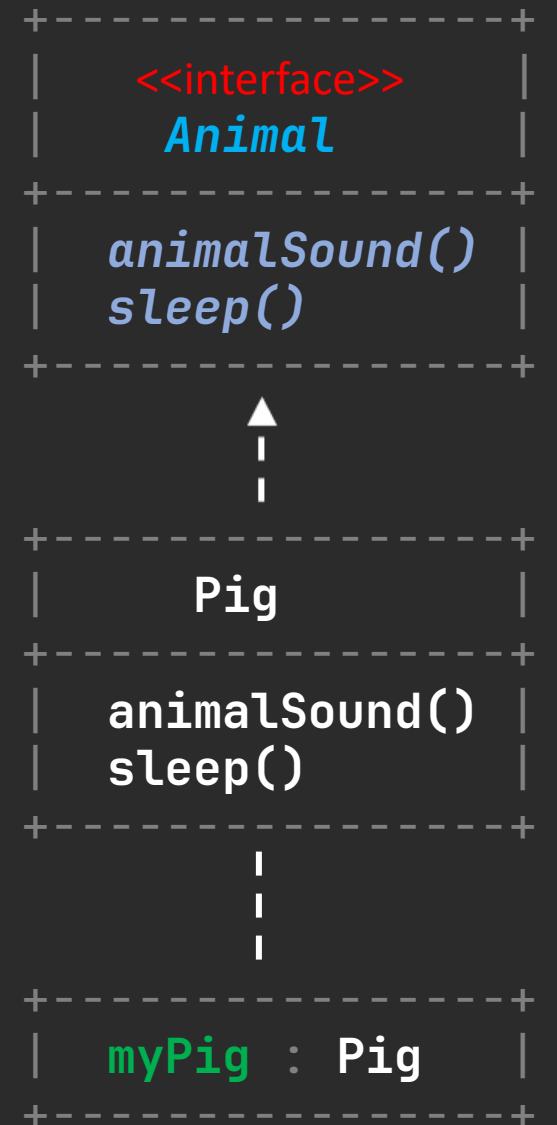


Interface

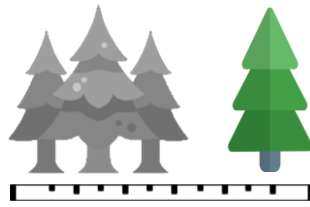


```
1 class test {
2   public static void main(String[] args) {
3       Pig myPig = new Pig();
4       myPig.animalSound();
5       myPig.sleep();
6   }
7 }
8
9 interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");
17     }
18     public void sleep() {
19         System.out.println("Zzz");
20     }
21 }
```

Wee wee



Interface

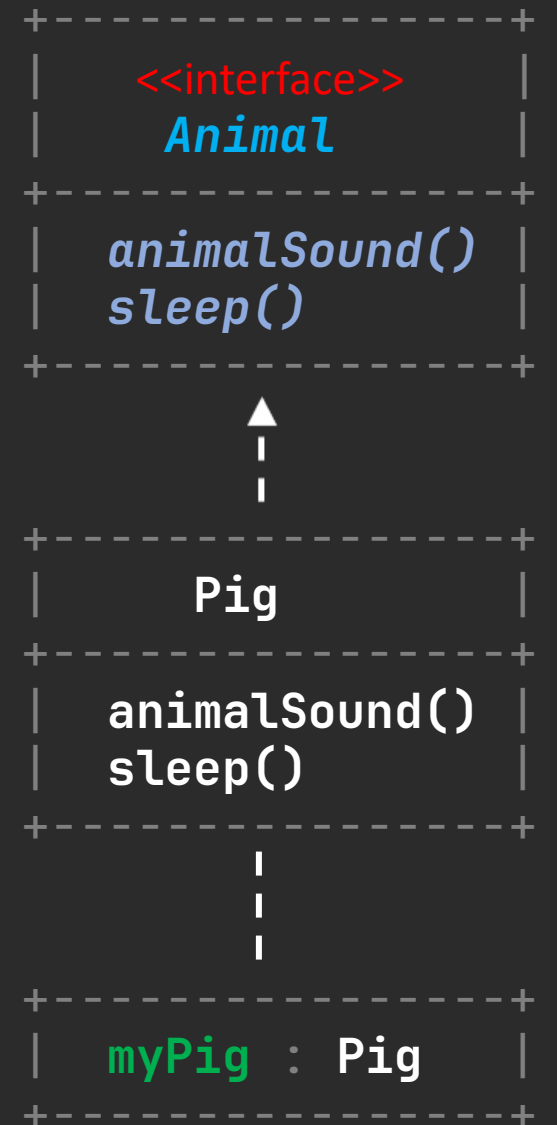


```

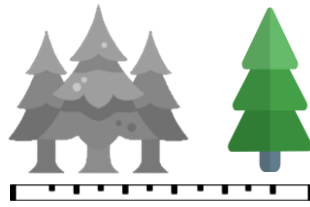
1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");
17     }
18     public void sleep() {
19         System.out.println("Zzz");
20     }
21 }

```

Wee wee



Interface



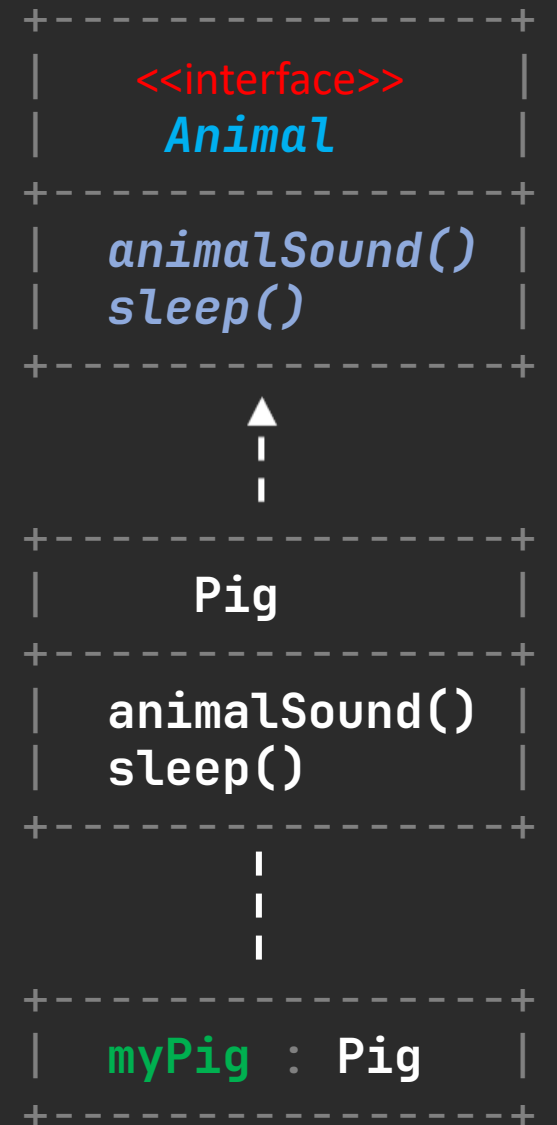
```

1 class test {
2     public static void main(String[] args) {
3         Pig myPig = new Pig();
4         myPig.animalSound();
5         myPig.sleep();
6     }
7 }
8
9 interface Animal {
10     void animalSound();
11     void sleep();
12 }
13
14 class Pig implements Animal {
15     public void animalSound() {
16         System.out.println("Wee wee");
17     }
18     public void sleep() {
19         System.out.println("Zzz");
20     }
21 }

```

Wee wee
Zzz

Wee wee
Zzz



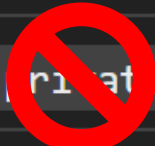
Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	All access modifiers allowed	
	<pre>int x = 0; int y = 1; int a = 3; int z = 2;</pre>	<pre>int x = 7;</pre>
Abstract Methods		
	<pre>void play(); void show(); void eat();</pre>	
Concrete Methods	Allowed with all access modifiers	
	<pre>void execute() {} void print() {} void stop() {} void log() {}</pre>	<pre>void print() {} void execute() {} void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	<code>static</code> <code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	Only <code>public static final</code> allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	
	<pre>void play(); void show(); void eat();</pre>	
Concrete Methods	Allowed with all access modifiers	
	<pre>void execute() {} void print() {} void stop() {} void log() {}</pre>	<pre>void print() {} void execute() {} void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	<code>static</code> <code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	Only <code>public static final</code> allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code> 	
	<pre>void play(); void show(); void eat();</pre>	
Concrete Methods	Allowed with all access modifiers	
	<pre>void execute() {} void print() {} void stop() {} void log() {}</pre>	<pre>void print() {} void execute() {} void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	<code>static</code> <code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	Only <code>public static final</code> allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>static</code> 	
	<pre>public abstract void play(); protected abstract void show(); abstract void eat();</pre>	
Concrete Methods	Allowed with all access modifiers	
	<pre>void execute() {} void print() {} void stop() {} void log() {}</pre>	<pre>void print() {} void execute() {} void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	<code>static</code> <code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	Only <code>public static final</code> allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>static</code> 	Only <code>public</code>
	<pre>public abstract void play(); protected abstract void show(); abstract void eat();</pre>	<pre>void show(); (public abstract implicitly)</pre>
Concrete Methods	Allowed with all access modifiers	
	<pre>void execute() {} void print() {} void stop() {} void log() {}</pre>	<pre>void print() {} void execute() {} void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	<code>static</code> <code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	Only <code>public static final</code> allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>static</code> 	Only <code>public</code>
	<pre>public abstract void play(); protected abstract void show(); abstract void eat();</pre>	<pre>void show(); (public abstract implicitly)</pre>
Concrete Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	
<code>static</code>	<pre>public static void execute() {} protected void print() {} void stop() {} private void log() {}</pre>	<pre>void print() {} void execute() {} void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables static	public , protected , default , private	Only public static final allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	public , protected , default ,	Only public
	<pre>public abstract void play(); protected abstract void show(); abstract void eat();</pre>	<pre> void show(); (public abstract implicitly)</pre>
Concrete Methods	public , protected , default , private	public default , public static , private
static	<pre>public static void execute() {} protected void print() {} void stop() {} private void log() {}</pre>	<pre> default void print() {} static void execute() {} private void log() {}</pre>

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Variables	<code>static</code> <code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	Only <code>public static final</code> allowed
	<pre>public static final int x = 0; protected int y = 1; int a = 3; private int z = 2;</pre>	<pre>public static final int x = 7;</pre>
Abstract Methods	<code>public</code> , <code>protected</code> , <code>default</code> ,	Only <code>public</code>
	<pre>public abstract void play(); protected abstract void show(); abstract void eat();</pre>	<pre>void show(); (public abstract implicitly)</pre>
Concrete Methods	<code>public</code> , <code>protected</code> , <code>default</code> , <code>private</code>	<code>public</code> <code>implicitly public</code> <code>private</code>
<code>static</code>	<pre>public static void execute() {} protected void print() {} void stop() {} private void log() {}</pre>	<pre>default void print() {} static void execute() {} void log() {}</pre>

Abstract Class vs. Interface

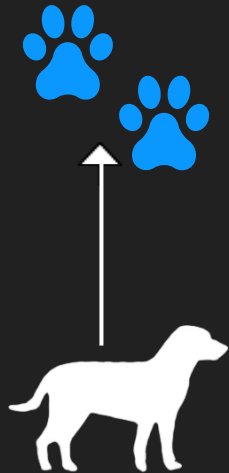
Feature	Abstract Class	Interface
Purpose		
Inheritance		
Constructors	 Yes	

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Purpose	For classes sharing common code among related classes (inheritance)	For enforcing a contract across unrelated classes (no inheritance relationship)
Inheritance	Supports Single inheritance (can extend only one class)	Supports Multiple inheritance (can implement many)
Constructors	 Yes but constructors are invoked during subclass instantiation through constructor chaining .	

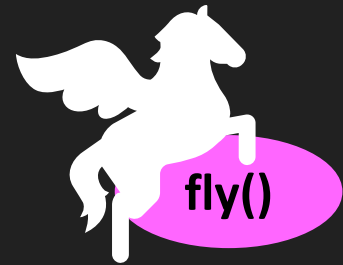
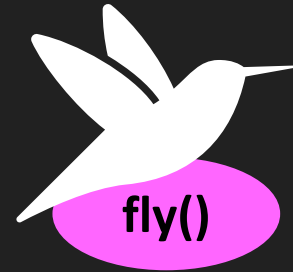
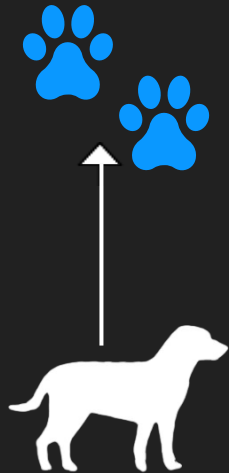
Abstract Class vs. Interface

Concept	Abstract Class (Mammal)	Interface (Fly)
Relationship	“is-a” relationship	relationship
Example	A Dog is a type of Mammal	



Abstract Class vs. Interface

Concept	Abstract Class (Mammal)	Interface (Fly)
Relationship	“is-a” relationship	“can-do” relationship
Example	A Dog is a type of Mammal	A Bird can-do what a Fly does



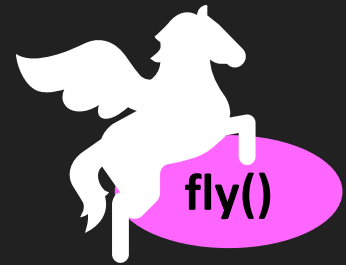
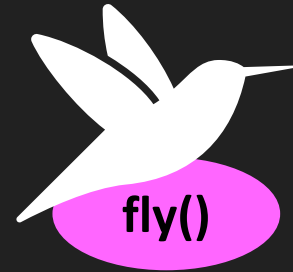
Abstract Class vs. Interface

Concept	Abstract Class (Mammal)	Interface (Fly)
Relationship	“is-a” relationship	“can-do” relationship
Example	A Dog is a type of Mammal	A Bird can-do what a Fly does



class Dog extends Mammal

- **class** is to **inherit**,



class bird implements fly
interface is to **implement**.